FLEX — Capital-Efficient Optimistic Bridges with On-Demand Security Bonds for Bitcoin

Sergio Demian Lerner* [1,2] and Ariel Futoransky† [1]

[1]Fairgate Labs
[2]Rootstock Labs

Abstract

This paper presents FLEX (Fraud proofs with Lightweight Escrows for eXits), a garbled circuit-based protocol designed to facilitate two-party disputes on Bitcoin without requiring permanent security bonds. FLEX enables conditional security deposits that are only activated in the event of a dispute, reducing the financial overhead for operators and challengers. The main goal of FLEX is to improve the capital efficiency of BitVM-based bridges in a permissioned challenge setting but can also be used to improve the security of any other fraud proof-based protocol such as payment channels. The paper also introduces enhancements that allow faster reimbursements in scenarios where one party's node is unavailable, while preserving security and minimizing race conditions.

1 Introduction

FLEX is a generic protocol designed to enhance the security and efficiency of blockchain fraud-proof systems, such as payment channels and optimistic bridges. It introduces conditional security deposits that are activated only in the event of a dispute, reducing financial overhead and improving scalability. FLEX can be applied to various blockchain applications, including decentralized exchanges, smart contracts, and sidechains, by optimizing the dispute cost structure. While FLEX is a general-purpose protocol, this work focuses on its application in the context of blockchain asset bridges.

Blockchain interoperability, particularly asset bridging, has garnered significant attention in the academic literature [1] [2,3]. The initial bridges [4] were designed to extend Bitcoin [5], relying on optimistic on-chain assertions and fraud proofs [6], but necessitated modifications to Bitcoin's consensus, such as the addition of new opcodes. BitVM [7] marked a significant advancement by proposing the first optimistic bridge for Bitcoin that does not require any changes to Bitcoin's consensus or the introduction of soft forks. The BitVM protocol is used to verify the burning of wrapped bitcoins on the side-system. The use of SNARKs [8] helps reduce both the amount of information posted on the Bitcoin blockchain and the amount of computation required to verify the burn event. BitVM has since inspired a range of protocols with different properties and dispute cost structures [9–12]. For instance, BitVM2 [13] offers a single on-chain round but incurs a high dispute cost, whereas BitVMX [11], though much cheaper in dispute costs, requires multiple rounds for resolution.

The feasibility of constructing an optimistic bridge [6] that is simultaneously fast, secure, and decentralized remains a subject of considerable debate in blockchain research. Numerous multi-party dispute resolution protocols, such as BoLD [14], Dave [15], and Optimism's [16] have been proposed to address

*sergio@fairgate.io
†futo@fairgate.io

this issue. The efficiency, security, and decentralization of a bridge protocol are inherently dependent on the cost of the dispute. Specifically, higher dispute costs necessitate larger security bonds, which must be recovered only if the dispute is resolved in favor of the party who posted the bond.

2 Sequential vs. Parallel Disputes

A central challenge in the design of optimistic bridges lies in their resistance to Denial-of-Service (DoS) attacks. If disputes can occur simultaneously and an unlimited number of challengers are permitted, a Sybil attacker could outspend the honest prover, thereby causing them to lose disputes. In such a scenario, the cost to defend against attacks becomes unbounded. High dispute costs, in turn, restrict participation to only those operators capable of securing large liquidity pools, thus centralizing the system. Conversely, if disputes are resolved sequentially, the finalization time increases as the number of challengers grows. The existing protocols present various trade-offs between dispute resolution time and associated costs. For example, some protocols feature logarithmic growth in resolution time relative to the number of challengers while maintaining a bounded cost.

3 Liveness violations and Misappropriation

In BitVM protocols, it is crucial not only for operators to act honestly but also for them to remain operational (i.e., "live"). Accordingly, we distinguish between two types of security bonds and their associated penalties.

Misappropriation penalties are imposed when a party deviates from honest behavior in an attempt to unlawfully extract funds from the bridge. In such cases, active challenges are required from the remaining operators, with each operator disputing separately, as BitVM does not support shared disputes. Additionally, a misappropriation attempt forces the challengers to lock their bonds for several timelock periods, which can range from 2 to over 20 periods depending on the specific BitVM protocol variant. Once the dispute is resolved, the malicious operator is penalized by slashing his security bond and/or blocking him from continuing to participate in the protocol, depending on the protocol specifics.

Liveness penalties are enforced when a party does not respond when required. Peg-ins cannot proceed unless all operators are available to co-sign the necessary transactions for subsequent peg-outs. If an operator is unavailable during a peg-in, economic damage is incurred by both other operators and end-users. Therefore, BitVM bridges must be designed to effectively penalize liveness violations. However, because liveness violations stem from unresponsive operators, penalties cannot rely on an on-demand security bond, which is only deposited when a peg-in or peg-out occurs. Instead, the security bond must be permanent and pre-existing, prior to these operations.

One of the costs associated with liveness violations during peg-outs is the reduced security. The risk of a misappropriation incentivizes watchtowers to employ high-availability servers. However, as the number of watchtowers increases, the associated risk decreases, which in turn reduces the expected loss (the probability of the event multiplied by the potential loss). This ultimately lowers the impact on the liveness security bonds.

To assess the remaining costs of liveness violations, we examine two cases: the first, where the protocol provides an optimistic path for rapid peg-out operations via a direct transfer to end-users or operators, facilitated by an aggregated n-of-n signature; and the second, where such a mechanism is not present.

3.1 Liveness Violation Cost with Optimistic Direct Transfers

Bridge protocols like Union [17] require continuous operator availability, as peg-outs enable an optimistic direct transfer path. This path has low latency: operators wait for a specified number of confirmation blocks of the burn transaction, after which the direct transfer is issued and immediately confirmed on the blockchain. Drop-offs are highly costly in this system, as they enforce a mandatory delay of one timelock period on each peg-out to allow challenges. This delay creates a financial burden, which must be covered either by peg-in fees or by the end-user initiating the peg-out. Therefore, even in cases where a party attempts to misappropriate funds, it is more beneficial for the system to keep the party active for peg-out facilitation. This is incentivized by maintaining a separate security bond to penalize liveness violations, which is not forfeited in the event of a misappropriation attempt.

The direct transfer path is only achievable in permissioned protocols, which is why it is not included in BitVM2 bridges.

When direct transfers are available, on-demand misappropriation security bonds can reduce the overall security bond requirements, which combine liveness and misappropriation penalties. However, the impact of this reduction varies depending on the total amount pegged in and out over time relative to the dispute cost. For example, if $100M is pegged out over a year after an operator drops off, but no peg-in occurs, and bitcoins staked yield a 3% return with a one-day timelock period per peg-out, the drop-off results in an opportunity cost of $8,219. If this cost is not passed to the end-user, the liveness security bond must be at least this amount to cover it. This value is comparable to the typical size of a misappropriation security bond in a BitVMX-based bridge.

3.2 Liveness Violation Cost without Optimistic Direct Transfers

In the absence of a direct transfer path, the forced peg-out delay becomes a sunk cost for the protocol, which is passed on to end-users. However, liveness violations are less impactful in this case. Liveness security bonds will generally be much lower than misappropriation security bonds.

Without direct transfers, on-demand peg-out security bonds can significantly reduce the financial burden on operators.

4 Prior Work

Historically, Bitcoin bridge designs relied on federated pegs or custodians, which necessitated trust in a majority of fixed parties (e.g., multisignature "pegnatories") to ensure the integrity of the system. These traditional approaches compromise the trustlessness of Bitcoin and rely on the reputation of functionaries, making them inefficient at scale. Recent advancements in off-chain verification, particularly the BitVM paradigm introduced in 2023, have enabled trust-minimized bridges by shifting the validation of cross-chain events onto Bitcoin itself through fraud proofs. This section reviews prominent BitVM-based Bitcoin bridges and related protocols, comparing their architectures, design trade-offs, and performance in terms of permissionlessness, fraud-proof mechanisms, trust models, and efficiency.

4.1 BitVM2 Bridge

Building on the BitVM concept, Linus et al. proposed BitVM2 as a generalized bridge protocol for Bitcoin sidechains or rollups [18] [13]. One key innovation is that it allows any party to act as a challenger, eliminating the need for a permissioned verifier role. As a result, the security model

becomes closer to open: any observer can submit a fraud proof if the operator (prover) attempts an invalid state transition or incorrect peg-out. The worst-case dispute now requires only three on-chain transactions, down from approximately 70 in the original BitVM1 [9] scheme, drastically reducing the cost and time required for fraud proofs. The trust model of BitVM2 is based on a 1-of-n honest minority assumption: as long as at least one altruistic or economically incentivized verifier is present, wrongful withdrawals will be detected. However, an N-of-N multisig is still required during protocol setup and peg-ins, meaning that BitVM2 still depends on the live participation of a pre-defined committee. The BitVM2 bridge had two versions [18] [13], each using a different technique to prove that the fronting transaction occurred in the Bitcoin canonical chain. The first uses superblocks (blocks with higher difficulty than the one established in the Bitcoin block header), the second uses the chain length. The first was shown to be insecure [19], and the second, not applicable to Bitcoin, as the Bitcoin protocol chooses the canonical chain as the one with the most cumulative difficulty, not the longest one. Both protocols assume a constant block difficulty, and fail to guarantee soundness if that is not the case. The BitVM2 bridge security could be strengthened by combining both approaches and increasing the number of superblocks required as a proof.

While BitVM2 demonstrates that a Bitcoin-secured bridge can be trust-minimized (without the need for a trusted custodian), the high dispute costs present a significant barrier to decentralization. Despite this, several BitVM2-based bridges have been developed, such as Clementine [20] and Strata [21], although the first drops the permissionless challenge feature.

4.2 Clementine (Citrea) Bridge

Clementine [20], one of the first BitVM2-based Bitcoin bridges, connects Bitcoin to the Citrea rollup. It verifies Citrea's state transitions optimistically via zero-knowledge validity proofs on Bitcoin, allowing BTC to be locked on the main chain and pegged into the rollup without the need for a centralized custodian. As with BitVM2, the security of Clementine is ensured by Bitcoin-scripted fraud proofs: any invalid state assertion from the rollup will be overturned on Bitcoin if challenged by an honest verifier. Clementine's latest design remains permissioned: only a watchtower can submit a fraud proof, and it requires counter-proofs in cases of invalid state assertions. It introduces several efficiency improvements, including the use of batch processing for withdrawals, where a single fraud proof can invalidate multiple pending withdrawals under a common collateral bond.

4.3 Strata (Alpen Labs) Bridge

Strata [21] is a Bitcoin bridge and sovereign rollup layer influenced by BitVM2. It retains the permissionless challenger model and introduces validity proofs for rollup state transitions, verified via SNARKs posted to Bitcoin. Strata adds an operator stake, which is forfeited if the operator misbehaves. While the same collateral can be staked in different disputes, the stake is permanent and locked during setup, so operators pay the financial cost of inactive collateral. Since Stata inherits from the BitVM2 Bridge the high cost of dispute, so too are the stake and its financial cost.

4.4 Rootstock Union Bridge

Rootstock [22] (RSK) has historically used a federated peg (the PowPeg) to bridge Bitcoin to its smart contract sidechain. The newer Union [17] bridge abandons the PowPeg in favor of a BitVM-powered trust-minimization approach. The Union protocol employs a virtual RISC-V CPU (BitVMX [11]) to execute SNARK verifiers, ensuring that Bitcoin can validate peg-out transactions from Rootstock with the assumption that at least one operator is honest. Unlike the PowPeg, Union requires only one

honest participant to secure the bridge, and its liquidity provider and watchtower roles are open to a wide range of participants, making it effectively permissionless and non-custodial.

4.5 Cardano's Cardinal

Cardano's Cardinal Bitcoin-Cardano bridge is another BitVMX-based protocol, allowing Bitcoin UTXOs to be wrapped as native tokens on Cardano and later redeemed on Bitcoin. Cardinal's security model is similar to other BitVMX-based systems, assuming at least one honest actor to ensure correct operation. However, Cardinal doesn't require operators to front transfers to users. Funds are directly accessible to end-users using the Transfer-of-Ownership Protocol (TOOP) [23]. This reduces notably the complexity of the protocol and increases its capital efficiency.

5 On-Demand Security Bonds in Bitcoin

In the protocols mentioned in the previous section, the cost of the dispute determines how fast the protocol can finalize and how decentralized it can be. If disputes are simultaneous, the amount of collective security deposits grows quadratically with the number of operators. Researchers have created multi-party dispute resolution protocols such as BoLD, Dave, and Optimism looking for good trade-offs between three properties: promptness, safety and decentralization. In the most efficient protocols, the security bonds are posted on-demand. However, until this work, on-demand bonds could not be achieved in Bitcoin. A key Bitcoin protocol limitation is that it is impossible to build a transaction DAG using emulated covenants that contains a security deposit transaction that consumes an input whose ID is unknown at construction time. A Bitcoin transaction ID depends on previous outputs consumed, using the transaction ID and its output index to uniquely reference it, so no fixed transaction can be pre-created to depend on an output of the security bond transaction. The `SIGHASH_NOINPUT` soft-fork can fix this problem, but there is no widespread consensus on its activation.

In summary, no existing BitVM protocol allows for on-demand security bonds in an incentive-compatible way. The BitVM1/BitVMX protocols required security bonds pre-deposited during setup, that must be maintained even if no peg-out is being processed. The BitVM2 Bridge introduced a mechanism to delay the challenger's security bond up to the peg-out time but not without additional problems. The challengers pre-pay the operator for the cost of the dispute, but without re-introducing an operator permanent security bond, this cost is sunk: the challengers cannot be reimbursed leading to a flaw in the incentive mechanism. The Clementine bridge based on BitVM2 requires permanent security bonds. FLEX presents a novel solution by enabling dynamic, on-demand peg-out security bonds, substantially improving capital efficiency and decentralization in garbled-circuit based bridges.

5.1 Garbled Circuits

The original Garbled Circuit (GC) protocol, introduced by Yao [24], is a two-party Secure Multi-Party Computation (MPC) scheme that allows parties to collaboratively evaluate a function over secret inputs provided by one or both parties, with only a single round of interaction. In this protocol, one party constructs an encrypted circuit, while the other party evaluates the circuit on encrypted inputs to produce either encrypted or decrypted outputs. To withstand a malicious circuit creator, the creator must demonstrate its correctness through either a succinct proof of computation integrity or by employing the cut-and-choose method.

In this work, we do not focus on any specific garbled circuit (GC) protocol variant or the off-chain costs; instead, we assume that a practical GC scheme can be developed.

Secrets in Yao's GC can be provided as encrypted inputs or embedded within the circuit itself by the circuit creator. However, when secrets are embedded in the circuit, special care must be taken to prevent their inadvertent disclosure while proving the correctness of the computation.

We use GC in a restricted setting: only one party—the GC creator—hides a single secret within the circuit, and when the circuit is evaluated, the creator provides the circuit's inputs. The secret is only revealed if the inputs to the computation satisfy specific, predefined conditions. This setup is commonly referred to as Conditional Disclosure of Secrets (CDS) [25]. While in typical CDS settings, the party providing the secret input is the receiver, in our case, the prover (the party initiating the dispute) provides the inputs. A significant advantage of revealing a secret upon satisfying a condition is that the secret can serve as the pre-image of a hash-lock, enabling a spending path to be triggered by arbitrary conditions. This property introduces a novel design space for BitVM protocols based on GC. By leveraging GC, we can resolve the issue of permanent security bonds in BitVM2 and enable dynamic security bonds, which is the core objective of the FLEX protocol.

6 The FLEX Protocol

We introduce the FLEX protocol, a garbled circuit-based solution designed to facilitate two-party peg-out disputes without requiring permanent security bonds. Security bonds are only deposited in the event of a dispute. This work focuses on the peg-out mechanism of a bridge, as it is the component requiring dispute resolution. We assume a permissioned challenge setting, where a committee of operators (such as liquidity providers or watchtowers) actively participates during peg-outs. For simplicity, we further assume that the validity of a peg-out is determined by a Burn Proof on the side-system, which is compressed using a SNARK.

In most blockchain systems, the security model of the side-system requires validating bridges to determine the canonical chain by receiving both proofs and counter-proofs. For instance, a Bitcoin validating bridge must compare forks with different cumulative difficulties to identify the canonical chain, and a proof-of-Stake validating bridge may require counter-proofs to prevent long-range attacks. While our permissioned challenge setting allows counter-proofs, they are excluded here for the sake of simplicity in our explanation. Figure 1 illustrates the simplified transaction Directed Acyclic Graph (DAG) involved in the protocol.

To enable counter-proofs, each circuit must evaluate two SNARKs, one provided by each party. The transaction graph is modified such that both parties first commit to their respective SNARKs (including the cumulative work as input). Subsequently, both parties provide the remaining circuit inputs by co-signing the other party's SNARK and cumulative work. Alice's circuit must only reveal the secret P_b if Alice's SNARK is invalid or if Bob's SNARK contains more cumulative work than Alice's SNARK. Similarly, Bob's circuit must reveal the secret P_a under the same conditions for Alice's SNARK.

6.1 Overview

The protocol involves two parties, Alice and Bob. Alice is assumed to be an operator of a bridge that fronts funds to an end-user, Carol. Alice will be reimbursed later by the protocol. Bob's role is to defend the protocol against any potential malicious actions by Alice. The protocol can be extended to support cases where both Alice and Bob act as operators. Carol, an end-user, requests a peg-out by burning bitcoins on a side-system. The protocol begins with a setup phase, in which Alice and Bob pre-sign eight transactions, creating a fixed transaction graph through a procedure known as emulated-covenant. The setup transactions are the following: `Peg-in`, `StartDispute`,`Reimbursement`,`AliceDeposit`,`BobDeposit`, `MissingAliceDeposit`,`MissingBobDeposit` and `AliceLost`.

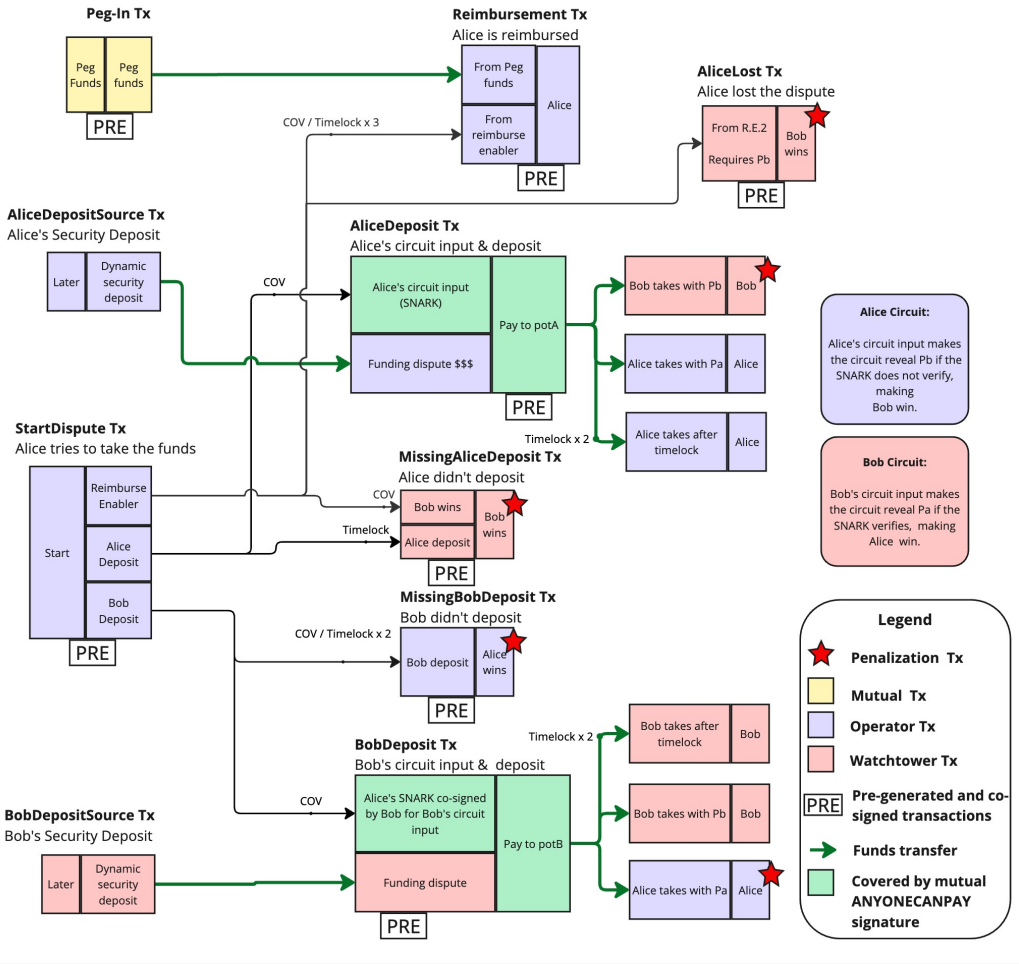

Figure 1: The basic FLEX transaction graph

There exists a high degree of symmetry in the transaction DAG, particularly regarding the roles of Alice and Bob. Of the transactions, only two are strictly related to Alice's role as a liquidity provider: `AliceLost` and `Reimbursement`.

6.2 Setup Phase

Alice and Bob exchange Lamport [26] public keys to serve as circuit inputs. Each party proves, in zero-knowledge, the correctness of the circuit and the correspondence between input labels and Lamport public keys. The input labels must be pre-images of the Lamport public keys, with the hash function being available as an opcode in Bitcoin's script. The proof can be interactive as the circuit is only valid for disputes between these two parties.

6.3 Peg-In Transaction

The `Peg-in` transaction holds the funds until Carol decides to peg-out by burning bitcoins in a side-system smart contract. The first output of the `Peg-in` transaction transfers the funds to either Carol or Alice. If Alice and Bob have agreed to pay Carol, they would transfer the funds directly to Carol's address. However, if Bob refuses to cooperate with Alice in the direct payment to Carol, Alice fronts the funds to Carol, and the reimbursement protocol is triggered.

6.4 Reimbursement Protocol

For Alice to initiate reimbursement, she must issue the `StartDispute` transaction. This transaction has three control outputs, which only require a minimal amount of bitcoin. The flows of significant amounts of funds are indicated by green arrows, while the connections between control input and outputs are shown in black. In general, most UTXOs consumed by BitVM protocols are control UTXOs. The output of the `StartDispute` transaction is termed the "Reimburse Enabler" which allows Alice to claim reimbursement via the `Reimbursement` transaction only after three timelock periods. All timelocks used in this protocol are relative. The remaining two outputs are allocated for Alice and Bob to deposit their security bonds and publish their circuit inputs. Upon issuance of the `StartDispute` transaction, three time-restricted events are triggered:

1. Alice deposits her security bond and publishes the SNARK circuit input OTS-signed by her in the `AliceDeposit` transaction, subject to penalty if not performed within the first timelock period.
2. Bob deposits his security bond and publishes the SNARK circuit input OTS-signed by both Alice and Bob in the `BobDeposit` transaction, subject to penalty if not performed within two timelock periods.
3. Alice can be reimbursed by issuing the `Reimbursement` transaction after three timelock periods.

If Alice does not issue the `AliceDeposit` transaction before the deadline, Bob can impose a penalty on Alice by issuing the `MissingAliceDeposit` transaction. Conversely, if Bob fails to issue the `BobDeposit` transaction within the deadline, Alice can issue the `MissingBobDeposit` transaction. The `BobDeposit` transaction can only occur if the `AliceDeposit` transaction has been broadcast, as Bob must first learn the SNARK program input to co-sign it.

6.5 Security Deposit Structure

The security deposits, namely `AliceDeposit` and `BobDeposit`, are prepared using partially pre-signed transactions that utilize the `SIGHASH_ANYONECANPAY` flag. The first transaction output contains a fixed amount, representing the protocol-agreed deposit, denoted by d per party. For a party to finalize the deposit transaction, it must add an input of at least d bitcoins. The funds for this input can be sourced from any UTXO available at the time of dispute. As illustrated in Figure 1, two transactions, `AliceDepositSource` and `BobDepositSource`, facilitate this action. Since `AliceDeposit` and `BobDeposit` transactions are fully constructed only when the dispute begins, their transaction IDs are not predetermined. However, the protocol does not require pre-creation of transactions that depend on the deposit transaction IDs. Instead, the security deposits are transferred to Taproot addresses, potA and potB, for Alice and Bob, respectively. Each address is associated with a Taproot tree containing three leaves, representing three potential spending paths, along with an unspendable internal key. In the `AliceDeposit` transaction, the three spending paths are constrained by the following script conditions:

1. Bob's signature verified by `OP_CHECKSIG`, along with the pre-image P_b such that $\text{Hash}(P_b) = V_b$, verified using `OP_HASH` and `OP_EQUAL`.
2. Alice's signature verified using `OP_CHECKSIG`, along with the pre-image P_a such that $\text{Hash}(P_a) = V_a$, verified using `OP_HASH` and `OP_EQUAL`.
3. Alice's signature is required for the current transaction, subject to a relative timelock expiry, enforced using `OP_CSV`.

The `BobDeposit` transaction is constructed in a manner analogous to Alice's deposit, except that it transfers the funds to the potB address. This address represents a Taproot tree, where the final leaf is associated with Bob, allowing him to recover the funds after the timelock expires.

6.6 Circuits Evaluation

During this phase, both Alice's and Bob's circuits are evaluated. Alice's SNARK is signed using Lamport signatures, revealing her circuit input wires. Alice uses her circuit's input wire labels as pre-images for the corresponding Lamport public keys. This ensures that the two states of a bit are never revealed simultaneously, as only one of the corresponding input labels is exposed. The mapping between Lamport signatures and input labels can be ensured through a SNARK during setup, which is efficient in terms of proving and verification cost. Bob uses the labels obtained to evaluate Alice's circuit. If the SNARK is invalid, Bob will obtain the secret output, P_b, which allows him to forfeit Alice's security deposit and prevent the reimbursement of the bridge funds.

Symmetrically, Bob's signs Alice's SNARK with his own Lamport keys, revealing his circuit input wire labels. Bob's circuit is evaluated by Alice. Bob circuit is designed so that, if Alice's SNARK is correct, a secret, P_a, is revealed. This secret is revealed by Bob's circuit under the opposite condition of Alice's circuit. Consequently, only one of the two secrets, P_a or P_b, will be revealed during the execution of the protocol. Let us define the relationship between P_a and P_b as follows: Hash(Pa) = V_a and Hash(P_b) = V_b, with values V_a and V_b exchanged between Alice and Bob during the setup phase.

6.7 Circuit Inputs Interdependence

After Alice publishes the `AliceDeposit` transaction containing Alice's SNARK, Bob is required to perform the same action. In Bob's transaction, he must include Alice's SNARK, which is signed by Alice using the Lamport One-Time Signature (OTS) scheme. In addition, Bob must co-sign the same message. Bob's Lamport signatures reveal the labels necessary for Alice to evaluate Bob's circuit. Note that because the `BobDeposit` transaction must include Alice's OTS signature for the SNARK as segregated witness, it almost doubles the witness size of the `AliceDeposit` transaction. The script executed in the `BobDeposit` transaction ensures that both signatures correspond to the same message.

6.8 Dispute Resolution

After both Alice and Bob evaluate each other's circuits, one of the two secrets, P_a or P_b, will be revealed. If P_a is revealed, it indicates that Bob has lost the dispute. In this case, Alice can claim both her own security deposit and Bob's security deposit by utilizing the appropriate spending paths. Conversely, if P_b is revealed, it indicates that Bob has won the dispute, and Bob can claim the security deposits by using P_b. Bob can also prevent the `Reimbursement` transaction from being executed by issuing the `AliceLost` transaction.

6.9 Faster Optimistic Peg-outs

In the simplified protocol, if Bob's node is down and cannot interact on-chain, the protocol waits for the full dispute period. Regardless of the nature of the dispute (fast, slow, or absent), Alice must wait for three timelock periods to be reimbursed. This introduces an undesirable financial cost. The following section describes an improved protocol that resolves these issues by enabling immediate direct transfers to Carol when both parties agree, and a resolution within two timelock periods when Bob is not operational but is not malicious.

6.10 Combining the Security Deposit and the setting of the Circuit Input

In the simplified protocol, we combine the publication and signing of the circuit input and the security deposit in a single transaction; however, this leads to incentive misalignment. Alice is required to pay for the SNARK publication before Bob commits to the security deposit, thus exposing her to risk. A 128-byte point-compressed SNARK with additional 32 bytes of witness data (160 bytes in total), when Lamport-signed, consumes 64 KB of witness data (15K vbytes [27]) and incurs a transaction fee (c) of approximately 75,000 satoshis (at current fee/vbyte rate) or $75 (at current USD/BTC rate). In the event of a significant increase in the Bitcoin transaction fees, this could pose a problem. We term this issue the Non-refundable Circuit Input Publication Problem (NCIPP), and in the next section we present three methods to mitigate it.

6.11 Resolving the Non-refundable Circuit Input Publication Problem (NCIPP)

The NCIPP arises because we made Alice be the first mover in our protocol. Alice must publish her circuit input before Bob commits to the security deposit. Given the high on-chain cost of the signed input, Alice is not refunded if Bob fails to provide his security deposit. In general, the first mover is typically at a disadvantage if the move incurs a high cost, as the second mover may choose to stop collaborating. However, we can assign the role of the first mover to Bob (the challenger), since Alice is already strongly incentivized to continue the dispute. If she does not, she forfeits her reimbursement and loses the bitcoins she has fronted to Carol. The minimum cost that Bob incurs is the opportunity cost of an initial security deposit, which is locked for only one timelock period. Meanwhile, Alice's liveness security bond should be sufficient to compensate Bob if Alice fails to continue the dispute. To address the NCPP problem we propose three methods:

1. Reimbursement from Bob's Persistent Security Deposit: In this method, Bob reimburses Alice's input publication cost (c sats) from a persistent security deposit. This deposit can be unique for disputes with Alice or shared among multiple parties.
2. Pre-deposit by Bob: Before Alice publishes her input, Bob pre-pays the cost (c sats) to Alice. This pre-deposit is added to the `AliceDeposit` transaction and subtracted from the `BobDeposit` transaction, ensuring both deposits are equal after completion.
3. Separation of Deposit and Input Publication: The deposit and input publication are split into two separate transactions, with the input publication occurring only after both security bond deposits have been performed.

6.12 Method 1: Persistent Security Deposit

Method 1 requires Bob to make a permanent security deposit, which, although minimal, incurs an ongoing cost. The deposit can be made either on the Bitcoin network or on a side-system. If the deposit is made on the Bitcoin network, the `StartDispute` transaction must also consume an additional input sourced from Bob's wallet. Alternatively, if the deposit is made on the side-system, a Bitcoin light client must be operational within the side-system contract to enable Alice to demonstrate that Bob did not perform the deposit. This is achieved by presenting the inclusion of the `MissingBobDeposit` transaction on the side-system. While this method ensures that Bob has a financial stake in the dispute, it introduces persistent costs associated with maintaining the deposit. Consequently, this method may not be optimal in situations where minimizing ongoing costs is a priority.

6.13 Method 2: Pre-deposit Transaction

Method 2 addresses the NCIPP by introducing a pre-deposit transaction as shown in Figure 2.

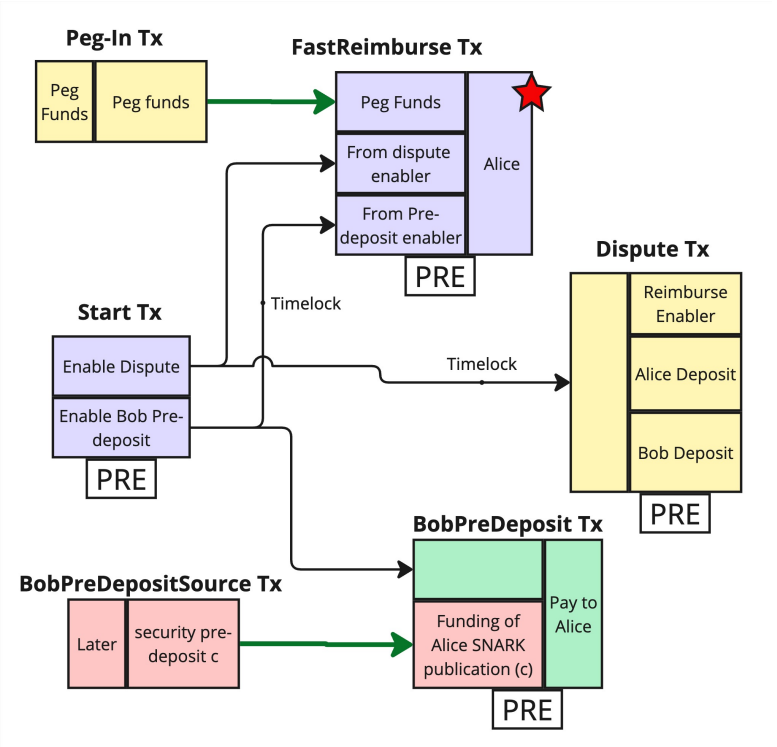

Figure 2: A modified protocol with Bob's predeposit transaction preventing spurious disputes

This method involves splitting the `StartDispute` transaction into two distinct phases: the `Start` transaction and the `Dispute` transaction. Upon issuing the `Start` transaction, Alice allows Bob to either initiate the dispute process or perform the pre-deposit. If Bob fails to execute the pre-deposit, Alice will wait for a timelock period to expire before issuing the `FastReimburse` transaction, allowing her to collect the funds. Bob will be unable to perform any further actions, as the `FastReimburse` transaction consumes the "Enable Dispute" output necessary to continue with the `Dispute` transaction. If Bob does issue the pre-deposit, the `BobPreDeposit` transaction invalidates the `FastReimburse` transaction, enabling him to proceed with the `Dispute` transaction at a later time. Importantly, if Bob does not issue the `Dispute` transaction, Alice is still able to proceed with the `Dispute` transaction independently.

6.14 Method 3: Separation of input publication and security deposit

We separate the deposit and input publication into two transactions. The input is only published after both parties have made their security bond deposits. As Alice's time window to post her security deposit is twice the time available for Bob, Alice can wait until Bob posts his, eliminating even minor incentive misalignment. This is the scheme chosen for the improved protocol presented in the following section.

7 FLEX2: Improved Protocol

We present an improved peg-out protocol based on an improved FLEX component. The improved protocol offers an optimistic direct payment to Carol when both parties agree, and an interlock method

is employed to prevent griefing attacks when disputes arise.

We first isolate the FLEX block from the bridge use case and make it generic. Then we extend it with additional features. We name this new building block the FLEX2 component, shown in Figure 3:

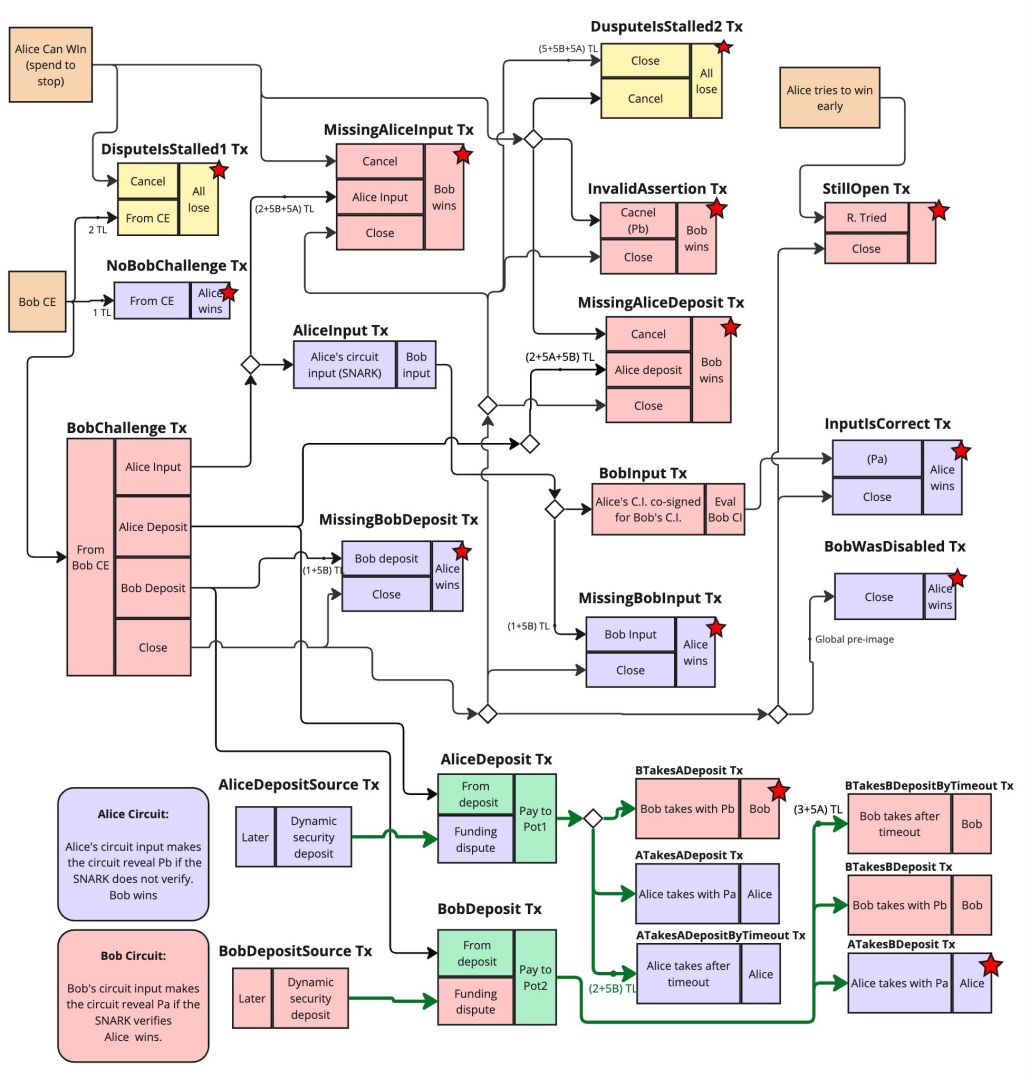

Figure 3: The Improved, Generalized and Isolated FLEX2 Component

This FLEX2 component involves two parties (Alice and Bob) where Alice is the asserter and Bob the challenger. Transactions that are issued by Alice are violet boxes, transactions issued by Bob are pink boxes, and transactions that any of the remaining asserters can issue are yellow boxes. The orange boxes are the component input UTXOs and the green areas are the parts covered by the covenant signatures in transactions where not all the parts are signed. The FLEX2 protocol also has two other parameters A and B that indicate additional acceptable delays for Alice and Bob to post their security bonds (Alice bond is also delayed by B), but we will not use these parameters in the peg-out protocol presented.

For the FLEX2 component to be used as part of larger multi-party protocol, we provide a method for external parties to close the dispute if the two parties do not interact. In this case, any of the other parties can issue the `DisputeIsStalled1` transaction. When Bob starts a challenge but then neither Alice nor Bob seems to continue with the protocol, the remaining parties can issue the `DisputeIsStalled2` transaction.

The FLEX2 protocol has 3 input UTXOs: **Alice Can Win**, **Bob Challenge Enabler**, and **Alice tries to win early**.

1. Alice Can Win. This is the output that allows Alice to win the dispute after it is closed. If Alice loses, this UTXO must be spent by the FLEX block.

2. Bob Challenge Enabler. This is the output that allows Bob to challenge and, therefore, to win. If this output is spent before the dispute starts, then Bob will be unable to challenge Alice.

3. Alice tries to win early. This is an output that Alice can dynamically make available during the progress of the dispute. Initially, this output is not present on the blockchain. As soon as it is available, a relative timelock allows Alice to win all the disputes (i.e. be reimbursed in a bridge) irrespective of the state of the ongoing disputes. If this output becomes available for spending while a dispute is still ongoing, it is Bob's responsibility to consume it as soon as possible, to prevent Alice from misappropriating funds. Bob prevents it by issuing the `StillOpen` transaction, which he can only issue if the dispute is still open. However, if this happens before Alice has issued an `AliceInput` transaction containing the circuit input, then Bob will not be able to claim the funds locked as security deposits, and each party will take his own deposit after the timelocks expire.

The normal order of transactions in the FLEX2 protocol is the following:

1. Alice posts her security bond with the `AliceDeposit` transaction.
2. Bob posts his security bond with the `BobDeposit` transaction.
3. Alice sends her circuit input with the `AliceInput` transaction.
4. Bob sends his circuit input (which must match Alice's) with the `BobInput` transaction.

If any party fails to proceed as expected, the other party can penalize them with transactions:

- `MissingAliceDeposit`, `MissingBobDeposit`, `MissingAliceInput`, `MissingBobInput`,
- `StillOpen`, `InvalidAssertion`, `NoBobChallenge`,
- `BTakesADeposit`, `ATakesBDeposit`.

Figure 4 presents the new pegout protocol using the FLEX2 component.

When the operator has won the disputes, he can issue the `TryEarlyRefund`/ `EarlyRefund` transaction pair or simply the `Refund` transaction. The early refund mechanism is a two-step process that allows Alice to be refunded quickly if Bob's node is down and neither cosigns the direct payment to Carol nor challenges Alice. To facilitate this, Alice waits for the expiration of one of Bob's timelock periods, issues the corresponding penalization transaction such `NoBobChallenge` or `MissingBobDeposit` to prevent race conditions, followed by the `TryEarlyRefund` transaction. After waiting for the second timelock period, Alice issues the `EarlyRefund` transaction to complete the process.

7.1 Summary

This paper introduces FLEX, a novel fraud-proof protocol based on Garbled Circuits (GC), designed to address the challenges associated with traditional BitVMX-based Bitcoin bridges. By enabling on-demand security bonds, FLEX reduces the capital inefficiencies inherent in prior systems that required pre-deposited security bonds for peg-outs, thus offering a more scalable and decentralized solution for cross-chain transactions. Through a comprehensive analysis, we identify critical issues in the design of optimistic bridges, particularly in ensuring Denial-of-Service (DoS) resistance and achieving the right balance between speed, security, and decentralization. Our work shows that the dispute cost

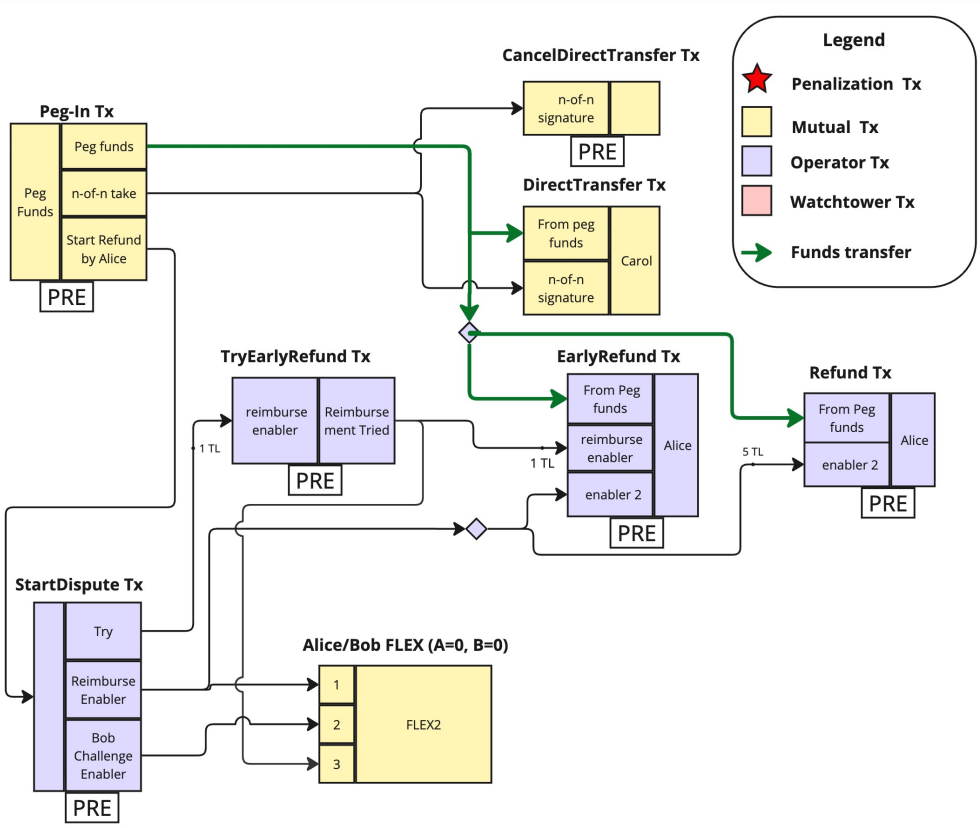

Figure 4: The Improved Peg-Out Protocol using FLEX2

directly impacts the decentralization and efficiency of a bridge, and we propose that on-demand bonds combined with sequential dispute resolution can maintain constant security bonds, which significantly reduces financial overheads. We further address the problem of ensuring fast disputes in case of non-collaborating parties. Through the evaluation of prior work, including BitVM2, Clementine, and Union bridges, we highlight the trade-offs between permissionlessness, dispute resolution costs, and system decentralization. FLEX offers a significant advancement in Bitcoin interoperability protocols by providing an incentive-compatible mechanism for on-demand security bond posting, thereby enhancing capital efficiency and decentralization in BitVM-based bridges.

References

[1] P. McCorry, C. Buckland, B. Yee, and D. Song, "Sok: Validating bridges as a scaling solution for blockchains," Cryptology ePrint Archive, Report 2021/1589, 2021, https://ia.cr/2021/1589.

[2] R. Belchior, A. Vasconcelos, S. Guerreiro, and M. Correia, "A Survey on Blockchain Interoperability: Past, Present, and Future Trends," *ACM Comput. Surv.*, vol. 54, no. 8, 2021.

[3] A. Zamyatin, M. Al-Bassam, D. Zindros, E. Kokoris-Kogias, P. Moreno-Sanchez, A. Kiayias, and W. J. Knottenbelt, "SoK: Communication Across Distributed Ledgers," in *Financial Cryptography and Data Security*, Berlin, Heidelberg, 2021, pp. 3–36.

[4] A. Back, M. Corallo, L. Dashjr, M. Friedenbach, G. Maxwell, A. Miller, A. Poelstra, J. Timón, and P. Wuille, "Enabling blockchain innovations with pegged sidechains," 2014, accessed: 2025-07-26. [Online]. Available: https://blockstream.com/sidechains.pdf

[5] S. Nakamoto, "Bitcoin: A peer-to-peer electronic cash system," 2008, accessed: 2025-07-26. [Online]. Available: https://bitcoin.org/bitcoin.pdf

[6] S. Jain, P. Saxena, F. Stephan, and J. Teutsch, "How to verify computation with a rational network," 2016, accessed: 2025-07-26. [Online]. Available: https://arxiv.org/abs/1606.05917

[7] R. Linus, "Bitvm: Compute anything on bitcoin," 2023. [Online]. Available: https://bitvm.org/bitvm.pdf

[8] J. Groth, "On the size of pairing-based non-interactive arguments," in *Proceedings of the 2016 ACM Conference on Computer and Communications Security (CCS)*. ACM, 2016, pp. 305–317. [Online]. Available: https://dl.acm.org/doi/10.1145/2976749.2978318

[9] L. Aumayr, Z. Avarikioti, R. Linus, M. Maffei, A. Pelosi, C. Stefo, and A. Zamyatin, "Bitvm: Quasi-turing complete computation on bitcoin," 2024, accessed: 2025-07-26. [Online]. Available: https://eprint.iacr.org/2024/1995.pdf

[10] A. Futoransky, E. Yago, and G. Guy, "BitSNARK & Grail bitcoin rails for unlimited smart contracts & scalability," 2024. [Online]. Available: https://assets-global.website-files.com/661e3b1622f7c56970b07a4c/662a7a89ce097389c876db57_BitSNARK__Grail.pdf

[11] S. D. Lerner, R. Amela, S. Mishra, M. Jonas, and J. Álvarez Cid-Fuentes, "BitVMX: A CPU for Universal Computation on Bitcoin," 2024. [Online]. Available: https://arxiv.org/abs/2405.06842

[12] A. L. Team, "Introducing snarknado," 2024, [Online; accessed 11-December-2024]. [Online]. Available: https://www.alpenlabs.io/blog/snarknado-practical-round-efficient-snark-verifier-on-bitcoin

[13] R. Linus, L. Aumayr, A. Zamyatin, A. Avarikioti, and M. Maffei, "Bitvm2: Bridging bitcoin to second layers," 2024, accessed: 2025-07-26. [Online]. Available: https://arxiv.org/abs/2405.06842

[14] M. M. Alvarez, H. Arneson, B. Berger, L. Bousfield, C. Buckland, Y. Edelman, E. W. Felten, D. Goldman, R. Jordan, M. Kelkar, A. Mamageishvili, H. Ng, A. Sanghi, V. Shoup, and T. Tsao, "Bold: Fast and cheap dispute resolution," 2024, accessed: 2025-07-26. [Online]. Available: https://arxiv.org/abs/2404.10491

[15] D. Nehab, G. C. de Paula, and A. Teixeira, "Dave: a decentralized, secure, and lively fraud-proof algorithm," 2024, accessed: 2025-07-26. [Online]. Available: https://arxiv.org/abs/2411.05463

[16] O. Foundation, "How malicious proposers exploit validator incentives in optimistic rollups," 2025, accessed: 2025-07-26. [Online]. Available: https://arxiv.org/html/2504.05094v1

[17] R. Amela, S. Mishra, S. D. Lerner, and J. Álvarez Cid-Fuentes, "Union: A trust-minimized bridge for rootstock," 2025, accessed: 2025-07-26. [Online]. Available: https://arxiv.org/abs/2501.07435

[18] R. Linus, "Bridging bitcoin to second layers - bitvm2," 2023, accessed: 2025-07-26. [Online]. Available: https://bitvm.org/bitvm_bridge.pdf

[19] S. D. Lerner, "A review of the bitvm2-based "linus24" bridge," 2024, accessed: 2025-07-26. [Online]. Available: https://www.fairgate.io/post/3-a-review-of-the-the-bitvm2-based-linus24-bridge

[20] E. Bal, L. Aumayr, A. İyidoğan, G. Scaffino, H. Karakuş, C. E. Aslan, and O. Litos, "Clementine: A collateral-efficient, trust-minimized, and scalable bitcoin bridge," 2025, accessed: 2025-07-26. [Online]. Available: https://arxiv.org/abs/2503.10185

[21] A. L. Team, "Introducing the strata bridge," 2024, [Online; accessed 11-December-2024]. [Online]. Available: https://www.alpenlabs.io/blog/introducing-the-strata-bridge

[22] S. D. Lerner, J. Álvarez Cid-Fuentes, J. Len, R. Fernández-València, P. Gallardo, N. Vescovo, R. Laprida, S. Mishra, F. Jinich, and D. Masini, "Rsk: A bitcoin sidechain with stateful smart contracts," Cryptology ePrint Archive, Paper 2022/684, 2022, accessed: 2025-07-26. [Online]. Available: https://eprint.iacr.org/2022/684

[23] A. Futoransky, F. Barbara, R. Fernández-València, G. Larotonda, and S. D. Lerner, "Toop: A transfer of ownership protocol over bitcoin," 2025, accessed: 2025-07-26. [Online]. Available: https://eprint.iacr.org/2025/964

[24] A. Yao, "Protocols for secure computations," 1982, accessed: 2025-07-26. [Online]. Available: https://dl.acm.org/doi/10.1145/800070.802212

[25] B. Applebaum, B. Arkis, P. Raykov, and P. N. Vasudevan, "Conditional disclosure of secrets: Amplification, closure, amortization, lower-bounds, and separations," Cryptology ePrint Archive, Paper 2017/164, 2017. [Online]. Available: https://eprint.iacr.org/2017/164

[26] L. Lamport, "Constructing digital signatures from a one way function," Tech. Rep., 1979.

[27] "Weight units — bitcoin wiki," 2021, [Online; accessed 11-December-2024]. [Online]. Available: https://en.bitcoin.it/wiki/Weight_units